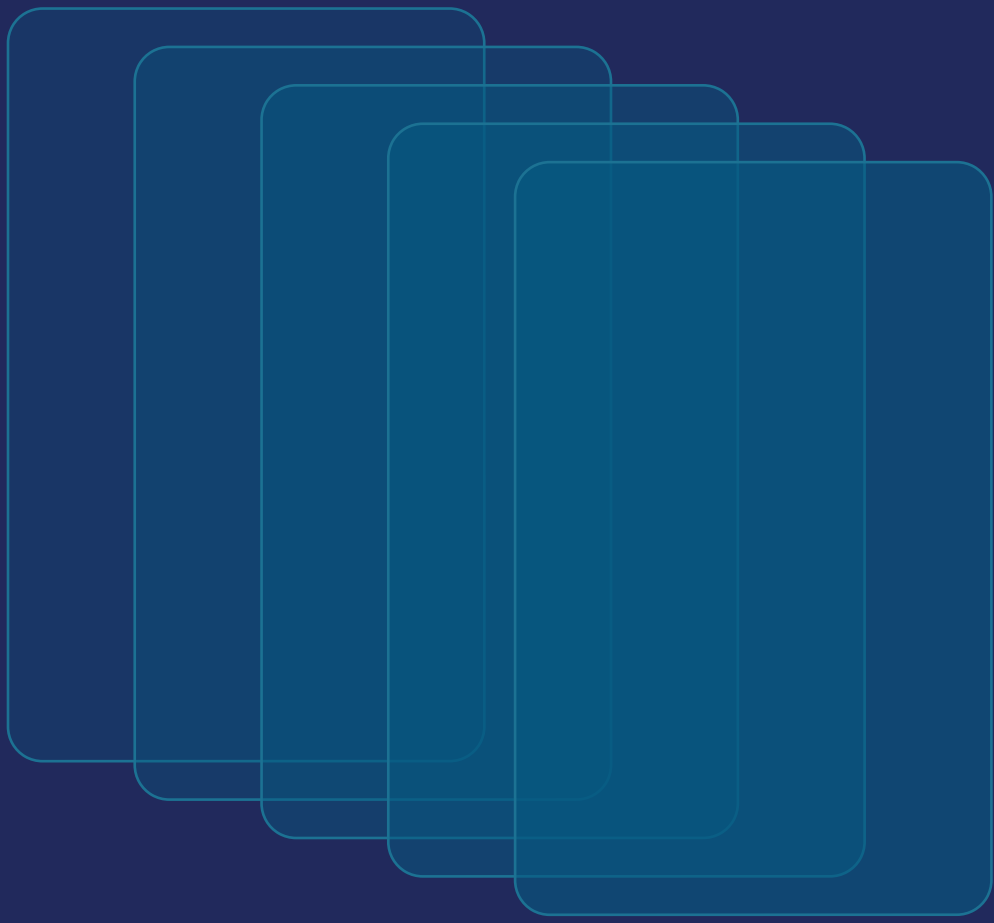
KUBERNETES TRAINING

## Module 6:

# ConfigMaps, Secrets, Init Containers & Multi-Container Patterns

Externalize configuration, protect sensitive data, control startup order, and design tightly coupled Pods.

2026 MODERNIZED EDITION



# From cluster operations to workload configuration

**Module 1**
Docker + image fundamentals

**Module 2**
Kubernetes architecture + kubectl

**Module 3**
Pods + Services

**Module 4 / 4B**
Namespaces + Deployments + Service types

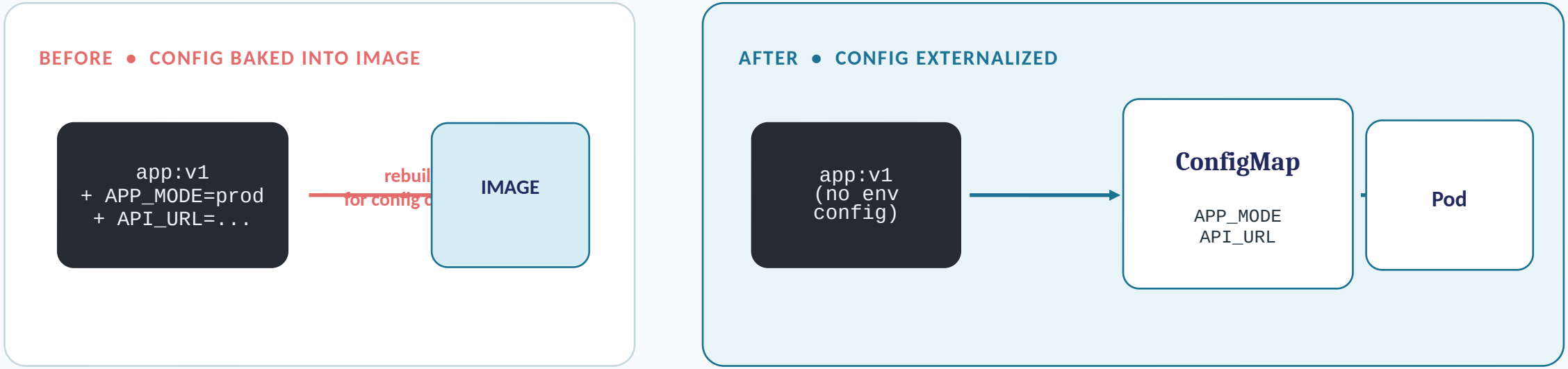
**Module 5**
Nodes + DaemonSets + Jobs + CronJobs

## THIS MODULE

- **A** ConfigMaps
- **B** Secrets
- **C** Init Containers
- **D** Sidecar / Adapter / Ambassador
- **E** **Native sidecars + lab + recap**

# Why externalize configuration?

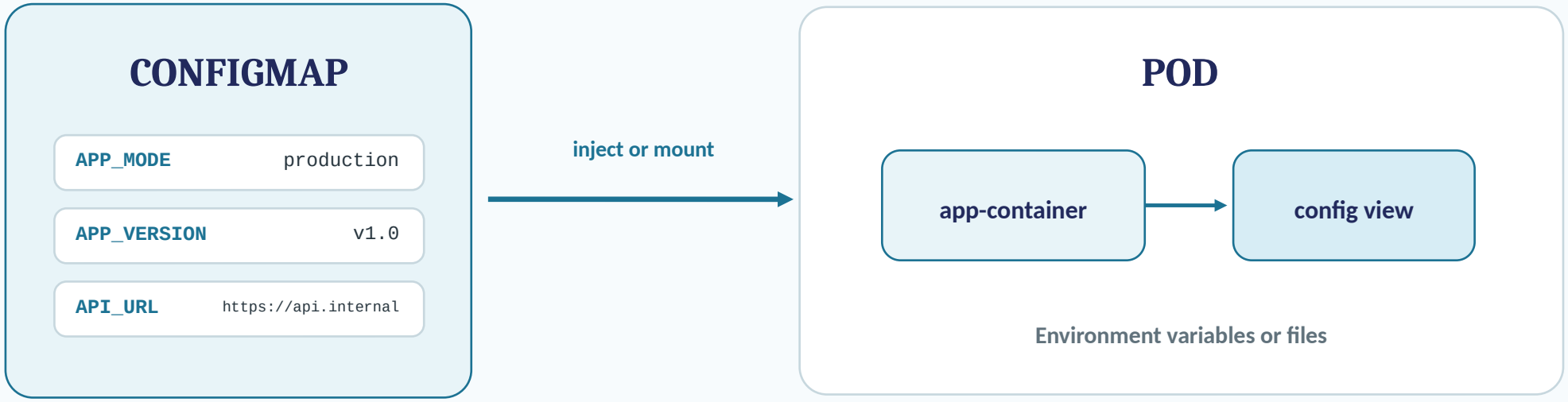
**The application image should describe the software, not every environment-specific setting.**




**Same image → different configuration per environment.**

# What is a ConfigMap?

**A ConfigMap stores non-sensitive key-value configuration separately from application code.**




### Use it for

URLs, flags, timeouts, non-secret settings.

### Do not use it for

Passwords, API keys, private keys, TLS secrets.

### Delivery options

envFrom / valueFrom / volume mount

# Create a ConfigMap — imperative approach

$ Terminal

```
kubectl create configmap my-config \
  --from-literal=APP_MODE=production \
  --from-literal=APP_VERSION=v1.0
```

OUTPUT

```
configmap/my-config created
```

$ Terminal

```
kubectl get configmaps
```

OUTPUT

```
NAME        DATA   AGE
my-config   2      10s
```

$ Terminal

```
kubectl describe configmap my-config
```

OUTPUT

```
APP_MODE:
----
production   # values shown in clear
```

# Create a ConfigMap — declarative approach

configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  APP_MODE: "production"
  APP_VERSION: "v1.0"
```

$ Terminal

```
kubectl apply -f configmap.yaml
```

OUTPUT

```
configmap/my-config created
```

## DECLARATIVE PAYOFF

1. **Versioned** — Store the manifest with the workload.
2. **Reviewable** — Changes are visible in Git/code review.
3. **Repeatable** — Recreate the same configuration reliably.
4. **Composable** — Reference the same object from multiple Pods.

# Use a ConfigMap as environment variables

env-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: env-pod
spec:
  containers:
    - name: busybox
      image: busybox:1.37
      command: ["sh", "-c", "echo APP_MODE=$APP_MODE && echo
APP_VERSION=$APP_VERSION && sleep 3600"]
      envFrom:
        - configMapRef:
            name: my-config
```

$ Terminal

```
kubectl apply -f env-pod.yaml
kubectl get pods
```

OUTPUT

```
pod/env-pod created

NAME      READY   STATUS    RESTARTS   AGE
env-pod   1/1     Running   0          4s
```

$ Terminal

```
kubectl logs env-pod
```

OUTPUT

```
APP_MODE=production
APP_VERSION=v1.0
```

# Use a ConfigMap as a mounted volume

volume-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: volume-pod
spec:
  containers:
    - name: busybox
      image: busybox:1.37
      command: ["sleep", "3600"]
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
  volumes:
    - name: config-volume
      configMap:
        name: config-volume
```

$ Terminal

```
kubectl create configmap config-volume \
  --from-file=app.properties
kubectl apply -f volume-pod.yaml
```

OUTPUT

```
configmap/config-volume created
pod/volume-pod created
```

$ Terminal

```
kubectl exec volume-pod -- cat /etc/config/app.properties
```

OUTPUT

```
APP_MODE=production
APP_VERSION=v1.0
```

# Modify a ConfigMap — volume vs env-var behavior

$ Terminal

```
kubectl edit configmap config-volume
```

Change the mounted file to: `APP_MODE=development` / `APP_VERSION=v2.0`

$ Terminal

```
kubectl exec volume-pod -- cat
/etc/config/app.properties
```

OUTPUT

```
APP_MODE=development
APP_VERSION=v2.0
```

**GOTCHA**

**Mounted ConfigMap files update after the kubelet syncs (seconds to ~2 min). Env vars are a snapshot — restart the Pod. Files mounted with subPath never update.**

# Delete a ConfigMap

$ Terminal

```
kubectl delete configmap my-config
```

OUTPUT

```
configmap "my-config" deleted from
default namespace
```

$ Terminal

```
kubectl get configmaps
```

OUTPUT

```
NAME              DATA   AGE
config-volume     1      3m
kube-root-ca.crt  1      2h
```

**IMPACT MODEL** Running Pods keep what they have. New Pods that need it fail: env → CreateContainerConfigError, volume → stuck in ContainerCreating. Use optional: true if the app can live without it.

# Why Secrets, not ConfigMaps, for sensitive data?

## ConfigMap

Non-sensitive settings

Examples: URLs, feature flags, timeouts

Readable as ordinary key-value data

SECURITY BOUNDARY

## Secret

Sensitive values

Examples: passwords, API keys, TLS material

Access can be restricted with RBAC

**COMMON MISCONCEPTION: base64 is ENCODING, not ENCRYPTION. It hides casual readability but is not a security control. Protect Secret access with RBAC and appropriate cluster storage controls.**

# Create a Secret — imperative approach

$ Terminal

```
kubectl create secret generic my-secret \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASSWORD=SuperSecret123
```

OUTPUT

```
secret/my-secret created
```

$ Terminal

```
kubectl get secrets
```

OUTPUT

```
NAME        TYPE     DATA   AGE
my-secret   Opaque   2      8s
```

$ Terminal

```
kubectl describe secret my-secret
```

OUTPUT

```
Type:  Opaque
DB_PASSWORD:  14 bytes
DB_USER:      5 bytes
```

# Create a Secret — declarative YAML

secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
type: Opaque
data:
  DB_USER: YWRtaW4=
  DB_PASSWORD: U3VwZXJTZWNyZXQxMjM=
```

$ Terminal

```
kubectl apply -f secret.yaml
```

OUTPUT

```
secret/my-secret created
```

## WHAT DID WE ENCODE?

base64

admin → **YWRtaW4=**

**Not encryption**

Anyone who can read the Secret data can decode it.
Treat the underlying value as sensitive.

Tip: stringData accepts clear text. Apply it with --server-side — a client-side apply copies it into an annotation in clear text.

# Use a Secret as environment variables

secret-env-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-env-pod
spec:
  containers:
    - name: busybox
      image: busybox:1.37
      command: ["sh", "-c", "echo DB_USER=$DB_USER && echo
DB_PASSWORD=$DB_PASSWORD && sleep 3600"]
      env:
        - name: DB_USER
          valueFrom:
            secretKeyRef:
              name: my-secret
              key: DB_USER
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: my-secret
              key: DB_PASSWORD
```

$ Terminal

```
kubectl apply -f secret-env-pod.yaml
```

OUTPUT

```
pod/secret-env-pod created
```

$ Terminal

```
kubectl logs secret-env-pod
```

OUTPUT

```
DB_USER=admin
DB_PASSWORD=SuperSecret123
```

Demo only — never log real credentials.

# Use a Secret as a mounted volume

secret-volume-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: secret-volume-pod
spec:
  containers:
    - name: busybox
      image: busybox:1.37
      command: ["sleep", "3600"]
      volumeMounts:
        - name: secret-volume
          mountPath: /etc/secret
  volumes:
    - name: secret-volume
      secret:
        secretName: secret-volume
```

$ Terminal

```
kubectl create secret generic \
  secret-volume --from-file=db-config
```

OUTPUT

```
secret/secret-volume created
```

$ Terminal

```
kubectl exec secret-volume-pod -- \
  cat /etc/secret/db-config
```

OUTPUT

```
DB_USER=admin
DB_PASSWORD=SuperSecret123
```

Secret volumes are tmpfs (RAM) mounts. Set defaultMode (e.g. 0440) and mount them read-only.

# Modify and delete a Secret

$ Terminal

```
kubectl edit secret secret-volume
```

OUTPUT

```
secret/secret-volume edited
```

$ Terminal

```
kubectl delete secret my-secret
```

OUTPUT

```
secret "my-secret" deleted
```

**MOUNTED FILE UPDATE**

A mounted Secret file updates after the kubelet syncs — about 70 s in our lab. No restart needed for files; env vars keep the old value until the Pod restarts.

**DELETION CAVEAT**

A deleted Secret cannot satisfy a new Pod that references it. Existing Pods retain the values already delivered to their containers; replacements are new consumers.

# Secrets are not revealed by kubectl describe

$ Terminal

```
kubectl describe secret my-secret
```

OUTPUT

```
Name:         my-secret
Namespace:    default
Labels:       <none>
Annotations:  <none>

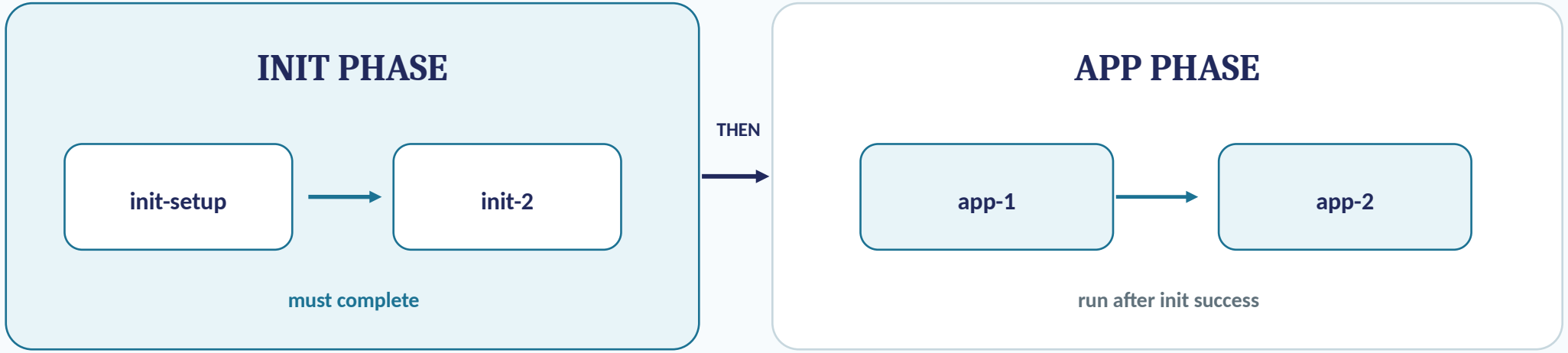
Type:  Opaque
Data
====
DB_PASSWORD:  14 bytes
DB_USER:      5 bytes
```

**PROTECTIVE BEHAVIOR**

describe shows keys and byte counts only. kubectl get secret -o yaml still shows base64 — and a client-side apply may have stored clear text in an annotation.

# What is an Init Container?

**Init containers perform setup work before application containers start. Multiple init containers run in order.**




### Setup

Files, config, dependency checks

### Order

init-1 → init-2 → app

### Failure

Pod stays in Init; app does not start

# Init Containers vs application containers

| DIMENSION | INIT CONTAINER | APPLICATION CONTAINER |
| --- | --- | --- |
| **Execution** | Sequential, before app start | Long-running app phase; containers can run together |
| **Purpose** | Setup / dependency checks / preparation | Serve the workload continuously |
| **Completion** | Must finish successfully | Usually remain running while the Pod is active |
| **Failure effect** | Blocks application startup | Affects the workload container / Pod lifecycle |

**Mental model: Init = “prepare, then hand off.” App = “serve while the Pod lives.”**

# Common use cases for Init Containers

### Wait for dependency

Poll until a database, service, or endpoint is reachable before the app starts.

### Prepare files

Generate config, permissions, templates, or seed data in a shared volume.

### Fetch prerequisites

Download a dependency or material the app should see before launch.

### Run one-time setup

Perform migrations or other setup tasks that must finish before serving traffic.

**Keep the init image/tooling separate from the application image when the setup utility is not part of the app runtime.**

# Init Container YAML — prepare, then start Nginx

init-container-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: init-container-pod
spec:
  initContainers:
    - name: init-setup
      image: busybox:1.37
      command: ["sh", "-c", "echo Initializing... && sleep 10"]
  containers:
    - name: nginx
      image: nginx:1.28-alpine
      ports:
        - containerPort: 80
```

$ Terminal

```
kubectl apply -f init-container-pod.yaml
```

OUTPUT

```
pod/init-container-pod created

NAME                READY   STATUS
init-container-pod   0/1   Init:0/1
```

$ Terminal

```
kubectl get pods
```

OUTPUT

```
NAME                READY   STATUS    RESTARTS
AGE
init-container-pod   1/1     Running   0
12s
```

`kubectl describe pod` shows an Init Containers section with `State: Terminated` and `Reason: Completed` after success.

# What happens when an Init Container fails?

failing-init-container-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: failing-init-container-pod
spec:
  initContainers:
    - name: failing-init
      image: busybox:1.37
      command: ["sh", "-c", "echo Failing Init Container... &&
exit 1"]
  containers:
    - name: nginx
      image: nginx:1.28-alpine
```

$ Terminal

```
kubectl get pods
```

OUTPUT

```
failing-init-container-pod   0/1   Init:Error
```

$ Terminal

```
kubectl logs failing-init-container-pod -c failing-init
```

OUTPUT

```
Failing Init Container...
```

## WHY THE APP DOES NOT START

The kubelet retries the init container with growing back-off (Init:Error → Init:CrashLoopBackOff). The app never starts until every init container succeeds.

# Init Container + shared volume — hand off prepared data

init-volume-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: init-volume-pod
spec:
  volumes:
    - name: shared-data
      emptyDir: {}
  initContainers:
    - name: write-config
      image: busybox:1.37
      command: ["sh", "-c", "echo 'APP_MODE=production' > /config/app.env"]
      volumeMounts:
        - name: shared-data
          mountPath: /config
  containers:
    - name: app
      image: busybox:1.37
      command: ["sh", "-c", "cat /config/app.env && sleep 3600"]
      volumeMounts:
        - name: shared-data
          mountPath: /config
```

$ Terminal

```
kubectl apply -f init-volume-pod.yaml
```

OUTPUT

```
pod/init-volume-pod created
```

$ Terminal

```
kubectl logs init-volume-pod
```

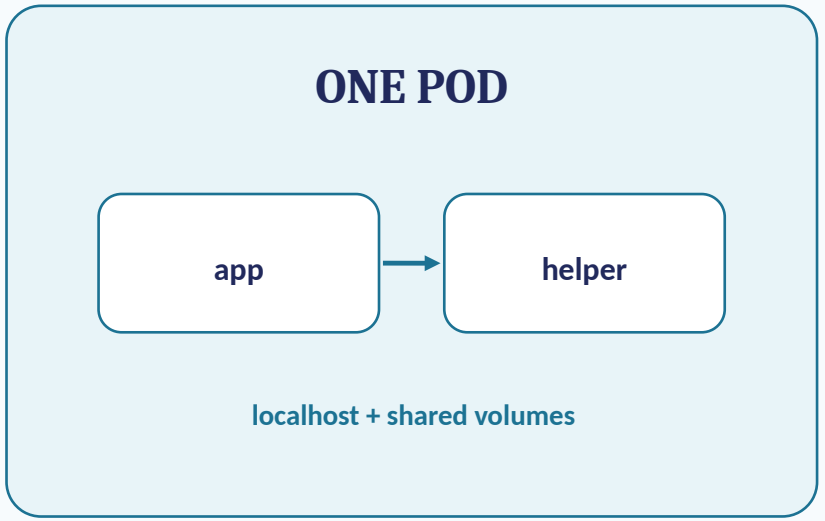
OUTPUT

```
APP_MODE=production
```

Init writes → shared `emptyDir` → app reads.

# Why multiple containers in one Pod?

**Containers in the same Pod share a network namespace and can share volumes, making them useful for tightly coupled helper processes.**




### SIDECAR

works alongside the app

### ADAPTER

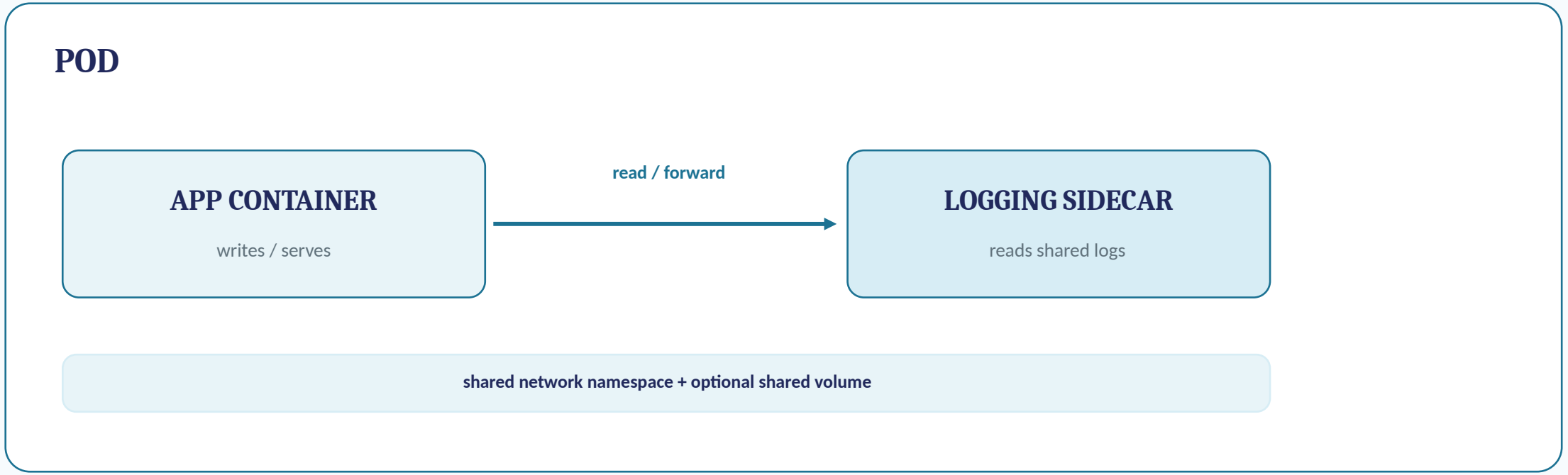
transforms app output

### AMBASSADOR

proxies communication

**Pattern choice follows the direction of the helper's responsibility: augment → transform → proxy.**

# Sidecar Pattern — a helper alongside the main app




**Sidecar = augment the app without changing its core process**

# Sidecar worked example — logging helper

sidecar-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: sidecar-pod
spec:
  volumes:
    - name: log-volume
      emptyDir: {}
  containers:
    - name: app-container
      image: nginx:1.28-alpine
      volumeMounts:
        - name: log-volume
          mountPath: /var/log/nginx
    - name: logging-sidecar
      image: busybox:1.37
      command: ["sh", "-c", "tail -F /var/log/nginx/access.log"]
      volumeMounts:
        - name: log-volume
          mountPath: /var/log/nginx
```

$ Terminal

```
kubectl exec sidecar-pod -c app-container -- wget -qO- http://localhost
```

OUTPUT

```
<!DOCTYPE html>
<html>...
</html>
```

$ Terminal

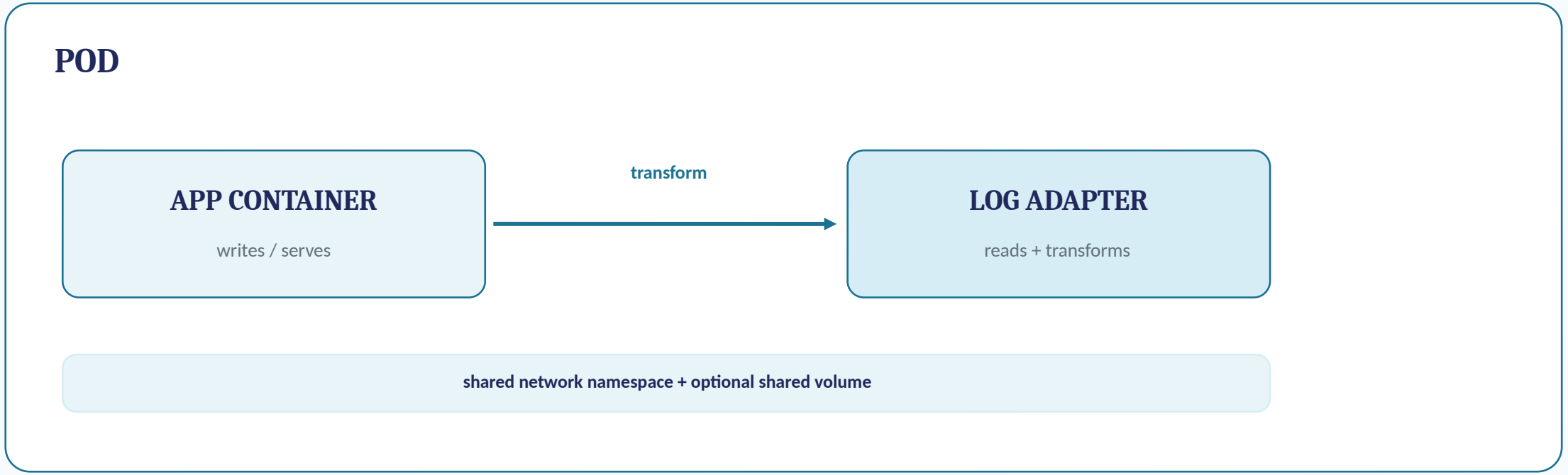
```
kubectl logs sidecar-pod -c logging-sidecar
```

OUTPUT

```
127.0.0.1 - - [29/Sep/2026:06:49:45 +0000]
"GET / HTTP/1.1" 200 615 "-" "Wget"
```

The app writes; the sidecar reads the same volume and emits the log stream.

# Adapter Pattern — transform the app's output




**Adapter = normalize or transform output into a standard form**

# Adapter worked example — uppercase log transformer

adapter-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: adapter-pod
spec:
  volumes:
    - name: log-volume
      emptyDir: {}
  containers:
    - name: app-container
      image: busybox:1.37
      command: ["sh", "-c", "echo 'error: something went wrong' > /log/output.log;
sleep 3600"]
      volumeMounts:
        - name: log-volume
          mountPath: /log
    - name: log-adapter
      image: busybox:1.37
      command: ["sh", "-c", "while true; do cat /log/output.log | tr a-z A-Z >
/log/processed.log; sleep 5; done"]
      volumeMounts:
        - name: log-volume
          mountPath: /log
```

$ Terminal

```
kubectl exec adapter-pod -c log-adapter -- cat /log/processed.log
```

OUTPUT

```
ERROR: SOMETHING WENT WRONG
```

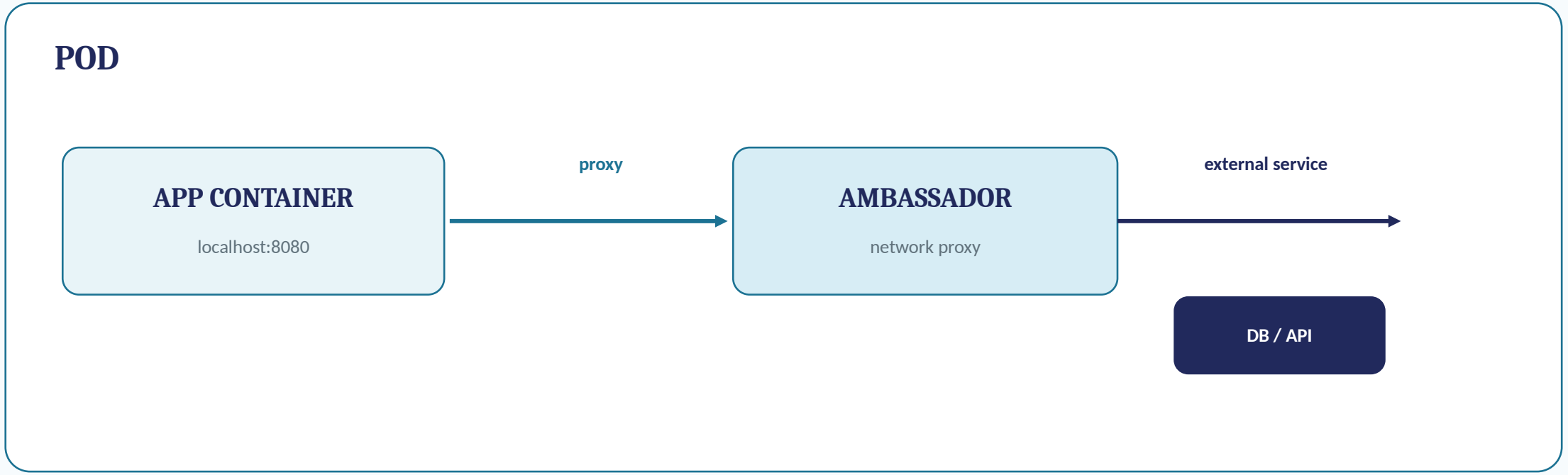
**TRANSFORMATION**

output.log
error: something went wrong

↓

processed.log
uppercase

# Ambassador Pattern — proxy network communication




**The app talks to localhost; the ambassador owns the network-facing connection.**

# Ambassador worked example — local proxy endpoint

ambassador-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: ambassador-pod
spec:
  containers:
    - name: app
      image: busybox:1.37
      command: ["sh", "-c", "while true; do wget -qO- localhost:8080; sleep 5; done"]
    - name: ambassador          # nginx reverse proxy
      image: nginx:1.28-alpine
      ports:
        - containerPort: 8080
      volumeMounts:
        - name: config
          mountPath: /etc/nginx/conf.d
  volumes:
    - name: config
      configMap:
        name: ambassador-config # proxy_pass http://orders-api
```

$ Terminal

```
kubectl logs ambassador-pod -c ambassador
```

OUTPUT

```
127.0.0.1 - - "GET / HTTP/1.1" 200 37 "Wget"
```

$ Terminal

```
k exec ambassador-pod -c app -- wget -O- localhost:8080
```

OUTPUT

```
{"service":"orders-api","orders":42}
```

**The app only knows localhost:8080. The ambassador owns the real address and timeouts — change them in the ConfigMap, not in the app.**

# Sidecar vs Adapter vs Ambassador

## SIDECAR

**Augment**

Logging, monitoring, proxy helpers

`App ↔ helper`

*"Do something beside me"*

## ADAPTER

**Transform**

Log parsing, format conversion

`App → helper → output`

*"Make my output usable"*

## AMBASSADOR

**Proxy**

API / DB network mediation

`App → proxy → external`

*"Talk outside for me"*

# Multi-Container Patterns in 2026 — Native Sidecars are stable

**Kubernetes v1.33+ supports native sidecars as restartable init containers. This gives the sidecar a defined startup order and Pod-lifetime behavior.**

LEGACY `spec.containers`

legacy sidecar

```
volumes:
  - name: log-volume
    emptyDir: {}
containers:
  - name: app
    image: nginx:1.28-alpine
    volumeMounts:
      - name: log-volume
        mountPath: /var/log/nginx
  - name: logging-sidecar
    image: busybox:1.37
    command: ["sh", "-c", "tail -F /var/log/nginx/access.log"]
    volumeMounts:
      - name: log-volume
        mountPath: /var/log/nginx
```

Works, but regular containers do not express a startup-order guarantee.

MODERN • v1.33+ `restartable initContainer`

native sidecar

```
volumes:
  - name: log-volume
    emptyDir: {}
initContainers:
  - name: logging-sidecar
    image: busybox:1.37
    restartPolicy: Always
    command: ["sh", "-c", "tail -F /var/log/nginx/access.log"]
    volumeMounts:
      - name: log-volume
        mountPath: /var/log/nginx
containers:
  - name: app
    image: nginx:1.28-alpine
    volumeMounts:
      - name: log-volume
        mountPath: /var/log/nginx
```

Starts before later app containers, runs for the Pod lifetime, and terminates automatically with the Pod.

**KEY SHIFT: “sidecar” is now a Kubernetes-native lifecycle construct, not only a design pattern.**

Official Kubernetes docs: Sidecar Containers • Stable since v1.33

# Hands-on lab — Try It Yourself

1 **Create ConfigMap**

$ Terminal

```
kubectl apply -f configmap.yaml
kubectl get configmaps
```

2 **Create Secret**

$ Terminal

```
kubectl apply -f secret.yaml
kubectl get secrets
```

3 **Build the Pod**

$ Terminal

```
kubectl apply -f lab-pod.yaml
kubectl get pod lab-pod
```

4 **Verify env + file**

$ Terminal

```
kubectl exec lab-pod -- env | grep
APP_MODE
kubectl exec lab-pod -- cat
/etc/secret/db-config
```

5 **Rebuild with native sideca**

$ Terminal

```
Replace logging container with
initContainers + restartPolicy: Always
kubectl apply -f native-sidecar-pod.yaml
```

6 **Inspect container lifecycle**

$ Terminal

```
kubectl get pod native-sidecar-pod
kubectl logs native-sidecar-pod -c
logging-sidecar
```

# Lab: ConfigMap updates — who sees them?

$ Terminal

```
kubectl apply -f cm-pod.yaml
# the same data three ways: envFrom,
# a volume, and a subPath file
```

OUTPUT

```
env      APP_MODE=production
volume   /etc/config/app.properties → ..data/…
subPath  /etc/app.properties (plain copy)
```

$ Terminal

```
kubectl apply -f config-volume-v2.yaml
kubectl patch configmap my-config … development
```

OUTPUT

```
configmap/config-volume configured
configmap/my-config patched
```

$ Terminal

```
kubectl exec cm-pod -- sh -c 'echo $APP_MODE;
  cat /etc/config/app.properties /etc/app.properties'
```

OUTPUT

```
production              ← env: snapshot
APP_MODE=development    ← volume: updated
APP_MODE=production     ← subPath: never updates
```

# Lab: missing and immutable ConfigMaps

$ Terminal

```
kubectl apply -f needs-config.yaml
# envFrom: my-config (deleted)
```

OUTPUT

```
needs-config   0/1   CreateContainerConfigError
Error: configmap "my-config" not found
```

$ Terminal

```
kubectl apply -f needs-volume.yaml
# volume from ConfigMap "nope"
```

OUTPUT

```
needs-volume   0/1   ContainerCreating
MountVolume.SetUp failed for volume "config" :
  configmap "nope" not found

# optional: true → the Pod starts without it
```

$ Terminal

```
kubectl apply -f app-config-v1.yaml
kubectl patch cm app-config-v1 …
```

OUTPUT

```
The ConfigMap "app-config-v1" is invalid: data:
Forbidden: field is immutable when `immutable` is set
```

# Lab: what base64 really protects

$ Terminal

```
kubectl get secret db-credentials -o \
  jsonpath='{.data.DB_PASSWORD}' \
  | base64 -d
```

OUTPUT

```
S3cure-Pa55w0rd!
# one command — base64 is encoding, not encryption
```

$ Terminal

```
kubectl apply -f secret.yaml
# (stringData, client-side apply)
kubectl get secret db-credentials \
  -o jsonpath='{.metadata.annotations}'
```

OUTPUT

```
{"kubectl.kubernetes.io/last-applied-configuration":
  "{…\"stringData\":{\"DB_PASSWORD\":
     \"S3cure-Pa55w0rd!\", …}"}

# the password, in clear text, in an annotation
```

$ Terminal

```
kubectl apply --server-side -f secret.yaml
```

OUTPUT

```
secret/db-credentials serverside-applied
# no last-applied annotation — nothing leaked
```

# Lab: mount Secrets the safe way

$ Terminal

```
kubectl apply -f secret-pod.yaml
# fsGroup: 1000, defaultMode: 0440,
# readOnly: true
```

OUTPUT

```
-r--r-----  root  1000  16  DB_PASSWORD
-r--r-----  root  1000   8  DB_USER
# the app's group can read, nobody else
```

$ Terminal

```
kubectl exec secret-pod -- \
  sh -c 'mount | grep /etc/db'
```

OUTPUT

```
tmpfs on /etc/db type tmpfs (ro,relatime,…)
# in memory only — never on the node's disk
```

$ Terminal

```
kubectl patch secret db-credentials … Rotated
# then poll the mounted file
```

OUTPUT

```
rotated after 70s    ← file updated, no restart
$DB_USER env var     ← unchanged until restart
```

# Lab: init containers that wait — and fail

$ Terminal

```
kubectl apply -f wait-for-db.yaml
kubectl get pod orders-app
```

OUTPUT

```
orders-app   0/1   Init:0/1   0   9s
wait-for-db: waiting for orders-db
```

$ Terminal

```
kubectl create service clusterip \
  orders-db --tcp=5432:5432
```

OUTPUT

```
service/orders-db created

orders-app   1/1   Running   0   10s
wait-for-db: orders-db is resolvable

# the Pod waited instead of crash-looping
```

$ Terminal

```
kubectl logs failing-init -c migrate
```

OUTPUT

```
failing-init   0/1   Init:Error → Init:CrashLoopBackOff
ERROR: relation users already exists
```

# Lab: a real ambassador — and startup order

$ Terminal

```
kubectl apply -f orders-api.yaml
kubectl apply -f ambassador-pod.yaml
```

OUTPUT

```
deployment.apps/orders-api created
configmap/ambassador-config created
```

$ Terminal

```
kubectl logs ambassador-pod -c app
```

OUTPUT

```
wget: can't connect … Connection refused
{"service":"orders-api","orders":42}
```

$ Terminal

```
kubectl apply -f ambassador-native.yaml
kubectl logs ambassador-native -c app
```

OUTPUT

```
{"service":"orders-api","orders":42}
# 1st request works — no refused
```

**Classic sidecars start in parallel with the app. A native sidecar with a startupProbe is ready before the app containers start.**

# Lab: native sidecars let Jobs finish

$ Terminal

```
kubectl apply -f job-classic-sidecar.yaml
kubectl apply -f job-native-sidecar.yaml
```

OUTPUT

```
job.batch/export-classic created
job.batch/export-native created
```

$ Terminal

```
kubectl get jobs
kubectl get pods -l 'job-name in (…)'
```

OUTPUT

```
export-classic   Running    0/1   (forever)
export-native    Complete   1/1   9s
export-classic-mns4r   1/2   NotReady
```

$ Terminal

```
kubectl logs job/export-native -c log-shipper
```

OUTPUT

```
exported batch 1
exported batch 2
exported batch 3
```

# Config & Secrets in production — 2026

## Versioned, immutable config

immutable: true (stable since v1.21) blocks accidental edits and saves the kubelet from watching the object.

Ship a new name per version (app-config-v2) — or let Kustomize's configMapGenerator add a hash — so every config change is a normal, reversible rollout.

## Secrets out of Git and annotations

base64 is not encryption. Turn on encryption at rest in the API server and limit get/list on Secrets with RBAC (Module 9).

Keep the real values in a secret manager and sync them in (External Secrets Operator, Secrets Store CSI driver). Apply Secret YAML with --server-side.

**2026 takeaway: config in Git, secrets in a manager, and a rollout for every change.**

Verified on Kubernetes v1.37: immutable ConfigMaps, server-side apply without the last-applied annotation

# ⚠ Common pitfalls — the gotchas to remember

### ConfigMap env vars go stale

Mounted files can update; env vars do not. Restart/recreate the Pod when env configuration changes.

### Base64 is not encryption

Anyone with access to Secret data can decode base64. Use RBAC and proper secret-management controls.

### Init failure blocks the app

Check `kubectl get pods`, `describe pod`, and `kubectl logs <pod> -c <init-name>`.

### Multi-container logs need `-c`

Use `kubectl logs <pod> -c <container-name>` to identify the correct process.

### Classic sidecar has no order guarantee

For lifecycle-aware sidecars, use native sidecar syntax in `initContainers`.

### Secrets leak via client-side apply

stringData lands in the last-applied annotation in clear text. Use kubectl apply --server-side.

Troubleshooting checklist • Runbook pp. 235, 242, 248, 253–255

# Module recap — configuration + lifecycle + patterns

### ConfigMap

Non-sensitive config; env or files

### Secret

Sensitive data; base64 ≠ encryption

### Env vs volume

Env is a snapshot; mounted data can update

### Init Container

Setup first; must complete successfully

### Sidecar

Helper alongside the app

### Adapter

Transforms app output

### Ambassador

Proxies network communication

### Native Sidecar

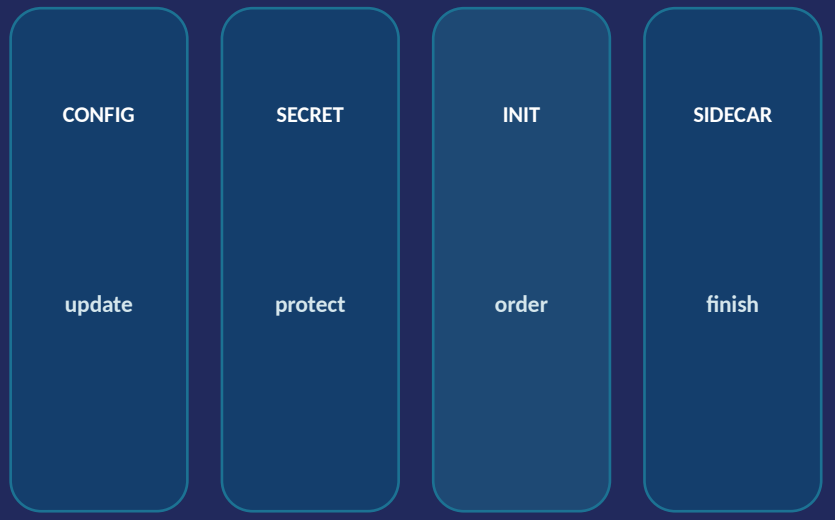
`initContainers` + `restartPolicy: Always`

**The practical sequence: externalize config → protect secrets → prepare with init → choose the right multi-container pattern.**

KNOWLEDGE CHECK

## 9 questions

# Before you move on




Real config, Secret, init and sidecar problems, with the terminal output you'd see. Answer key on the last quiz slide.

# Question 1 of 9

**You changed LOG_LEVEL in a ConfigMap an hour ago. The running Pods, which use envFrom, still log at the old level. Why?**

$ Scenario

```
k exec api-7c9-x2 -- printenv LOG_LEVEL
```

OUTPUT

```
info

# ConfigMap now says: debug
```

Think about

When are env vars read?

CHOOSE ONE

A. The kubelet sync takes up to 24 hours
B. The ConfigMap is immutable
C. Env vars are set once at container start — roll the Pods (kubectl rollout restart)
D. envFrom only reads keys in capital letters

# Question 2 of 9

**A ConfigMap file mounted with subPath never shows the new values, while other mounted files do. What's going on?**

$ Scenario

```
k exec web -- cat /etc/nginx/nginx.conf
```

OUTPUT

```
worker_processes 1;   # old value

# volumeMounts: subPath: nginx.conf
```

Think about

Symlink vs copy

CHOOSE ONE

A. subPath mounts are a copy that never updates — mount the directory or restart
B. The file is cached by nginx
C. ConfigMaps only update once a day
D. subPath needs readOnly: true to update

# Question 3 of 9

**A new Pod is stuck. What's the fastest correct fix?**

**$ Scenario**

```
k get pod api-5d8-q2
```

**OUTPUT**

```
NAME        READY   STATUS
api-5d8-q2  0/1     CreateContainerConfigError
Error: configmap "api-config" not found
```

**Think about**

Read the error literally

**CHOOSE ONE**

A. Delete the Pod and let it restart
B. Increase the Pod's memory limit
C. Pull the image again
D. Create the api-config ConfigMap (or mark the reference optional)

# Question 4 of 9

**A teammate wants to commit this Secret to Git: "it's base64, so it's encrypted". What do you say?**

$ Scenario

```
cat secret.yaml
```

OUTPUT

```
data:
  DB_PASSWORD: UzNjdXJlLVBhNTV3MHJkIQ==
```

Think about

Encoding vs encryption

CHOOSE ONE

A. Fine — Kubernetes encrypts base64 values
B. No — anyone can decode base64; keep real values in a secret manager
C. Fine, as long as the repository is private
D. Only if the key names are base64 too

# Question 5 of 9

**You applied a Secret with stringData, and anyone can read the password below. How do you prevent this?**

$ Scenario

```
k get secret db -o yaml | grep -A1
annotations
```

OUTPUT

```
kubectl.kubernetes.io/last-applied-configuration:
  {…"stringData":{"DB_PASSWORD":"S3cure…"}}
```

Think about

Which apply writes that annotation?

CHOOSE ONE

A. Use data instead of stringData
B. Delete the annotation after every apply
C. Apply Secrets with kubectl apply --server-side (or create them with kubectl create secret)
D. Add type: Encrypted

# Question 6 of 9

**A Pod has been in Init:0/1 for ten minutes. What's the most likely problem?**

**$ Scenario**

```
k logs orders-app -c wait-for-db
```

**OUTPUT**

```
waiting for orders-db
waiting for orders-db
waiting for orders-db
```

**Think about**

What is the init container waiting for?

**CHOOSE ONE**

A. The Service orders-db doesn't exist yet (or has a different name)
B. The app container's image is wrong
C. Init containers always take 10 minutes
D. The node is out of memory

# Question 7 of 9

**A Job's log-shipper sidecar is in spec.containers. The task finished an hour ago, but the Job is still Running. Fix?**

$ Scenario

```
k get pods -l job-name=export
```

OUTPUT

```
NAME           READY   STATUS
export-mns4r   1/2     NotReady
```

Think about

Who stops a native sidecar?

CHOOSE ONE

A. Add backoffLimit: 0
B. Delete the Job by hand after each run
C. Set activeDeadlineSeconds on the sidecar
D. Move the sidecar to initContainers with restartPolicy: Always

# Question 8 of 9

**The app gets "connection refused" on localhost:8080 at start-up because its proxy sidecar starts slower. Best fix?**

**$ Scenario**

```
k logs pay-7f9 -c app | head -2
```

**OUTPUT**

```
wget: can't connect to remote host
(127.0.0.1): Connection refused
```

**Think about**

Startup order

**CHOOSE ONE**

A. Add sleep 10 to the app's command
B. Make the proxy a native sidecar with a startupProbe
C. Use a NodePort Service
D. Put both containers in separate Pods

# Question 9 of 9

**Your logging system only accepts JSON, but a legacy app writes plain text. Which pattern fits?**

**$ Scenario**

```
k exec legacy -- tail -1 /logs/app.log
```

**OUTPUT**

```
06:49:48 error: payment failed order=6841
```

**Think about**

Transform, proxy or prepare?

**CHOOSE ONE**

A. Ambassador
B. Init container
C. Adapter
D. DaemonSet

# Answer key

Score yourself

9 questions
1 point each

ANSWERS

```
Q1  C  env is a snapshot — roll Pods → slide 34 • Step 4
Q2  A  subPath never updates         → slide 34 • Step 4
Q3  D  create it or mark optional    → slide 35 • Step 5
Q4  B  base64 is not encryption      → slide 36 • Step 6
Q5  C  kubectl apply --server-side   → slide 36 • Step 6
Q6  A  the Service doesn't exist     → slide 38 • Step 9
Q7  D  native sidecar                → slide 40 • Step 15
Q8  B  native sidecar + startupProbe → slide 39 • Step 14
Q9  C  Adapter                       → slide 28 • Step 13
```

| | |
|---|---|
| 8–9 | Module 7 ready |
| 6–7 | re-read misses |
| ≤ 5 | redo Steps 3–15 |

MODULE 6 COMPLETE

## Next: Module 7

# Resource Management, Probes & Scheduling




You now know how configuration enters a Pod, how startup work is staged, and how tightly coupled containers can cooperate.